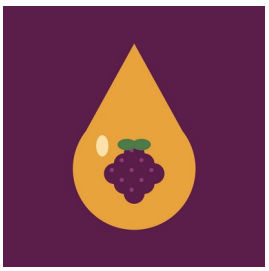
[NOMBRE DE LA INSTITUCIÓN / UNIVERSIDAD]
[Facultad / Carrera]



MORA & MIEL

Pastelería artesanal

Reingeniería, gestión de servicios nativos y auditoría de código en repositorio público

Actividad Integradora 2

Estudiante: Cinthia Marisol Guamán Montaño
Asignatura: Desarrollo de Aplicaciones Móviles
Docente: **[Nombre del docente]**
Fecha de entrega: **[dd/mm/2026]**

NOTA SOBRE EL PROYECTO BASE

Sí se utiliza el proyecto de la Actividad Integradora 1. Esta entrega es la reingeniería de la aplicación **“Dulce Antojo”** (catálogo de postres con dos pantallas, navegación con rutas nombradas y paso del objeto `Postre` como parámetro) desarrollada en la Actividad Integradora 1.

Sobre esa base se aplicaron todas las mejoras: rebranding a **“Mora & Miel”**, cambio de identificador de paquete, nueva arquitectura por capas con **Provider** y la integración de tres servicios nativos (conectividad, cámara con permisos y notificaciones locales). El primer commit del repositorio contiene el código original de la Actividad 1 para que la evolución sea trazable.

ÍNDICE

Sección I: Justificación del Rebranding y Cuadro Comparativo

1.1 Nueva propuesta de valor
1.2 Justificación del cambio de marca
1.3 Cuadro comparativo (antes vs. después)
1.4 Comparación de assets · 1.5 Paleta · 1.6 Interfaz

Sección II: Arquitectura y Patrones de Desarrollo

2.1 Responsabilidad de cada capa
2.2 Cómo se logra el desacoplamiento
2.3 Flujo de una reserva sin conexión

Sección III: Explicación de Código y Manejo de Excepciones

3.1 Detección reactiva de red
3.2 Estrategia de caché y sincronización
3.3 Manejo dinámico de permisos
3.4 Captura, recorte y compresión
3.5 Servicio de notificaciones locales
3.6 Resumen de excepciones controladas

Sección IV: Evidencias de Pruebas (QA) y Enlace al Repositorio Público

4.1 Repositorio público
4.2 Historial de ramas
4.3 Casos de prueba
4.4 Capturas de funcionamiento

Sección V: Conclusiones y Lecciones Aprendidas

5.1 Ciclo de vida del dispositivo
5.2 Concurrencia y asincronía
5.3 Control de versiones
5.4 Conclusión general
Declaración de uso de Inteligencia Artificial

Sección I: Justificación del Rebranding y Cuadro Comparativo

1.1 Nueva propuesta de valor

Dulce Antojo era un prototipo (MVP) estático: mostraba cuatro postres y un formulario que solo devolvía un mensaje a la pantalla anterior. No guardaba nada, no funcionaba sin internet y su identidad visual era el rosado por defecto de Flutter con el ícono genérico del framework.

Mora & Miel se plantea como una **pastelería artesanal con pedidos personalizados**. La propuesta de valor se resume en tres promesas, cada una respaldada por un servicio nativo:

- **“Tu antojo, aunque no tengas señal”:** el catálogo se guarda en caché y las reservas hechas sin conexión se envían solas al volver internet (servicio de conectividad).
- **“Muéstranos el postre que imaginas”:** el cliente adjunta una foto de referencia desde la cámara o la galería; la app la recorta y comprime antes de guardarla (cámara + permisos).
- **“Te avisamos al instante”:** notificaciones locales confirman la reserva, avisan si quedó pendiente y cuando se sincronizó (notificaciones).

1.2 Justificación del cambio de marca

- **Nombre más distintivo y memorable.** “Dulce Antojo” es una expresión genérica muy usada por negocios de repostería; “Mora & Miel” combina dos ingredientes naturales, comunica lo artesanal y es fácil de registrar como marca y como dominio.
- **Coherencia con el nuevo catálogo.** Los postres se reformularon con mora y miel (tiramisú de mora, cupcakes de miel, cheesecake de frutos rojos, tres leches con miel), así nombre, producto e imagen cuentan la misma historia.
- **Identidad visual propia.** El rosado genérico se reemplazó por una paleta cálida y diferenciada: el morado de la mora transmite calidad “premium” y el dorado de la miel, calidez y sabor. El emblema (una gota de miel que contiene una mora) resume la marca y se reconoce incluso a 48 px.
- **Profesionalización técnica.** Se reemplazó el identificador `com.dulceantojo.dulce_antojo` por `com.moraymiel.app` (corto, sin guiones bajos y consistente en Android e iOS), requisito para publicar en tiendas como una aplicación nueva.
- **Mantenibilidad.** Antes cada pantalla repetía `Colors.pink`; ahora todos los colores salen de `AppColors` y `AppTheme`, de modo que un cambio de marca futuro se hace en un solo archivo.

1.3 Cuadro comparativo (antes vs. después)

Elemento	Antes · Dulce Antojo (AI 1)	Después · Mora & Miel (AI 2)
Nombre comercial	Dulce Antojo	**Mora & Miel**
Package Android	`com.dulceantojo.dulce_antojo`	`com.moraymiel.app`
Bundle ID iOS	`com.dulceantojo.dulceAntojo`	`com.moraymiel.app`
Paquete Dart / versión	`dulce_antojo` 1.0.0+1	`mora_y_miel` 2.0.0+2
Ícono de la app	Ícono por defecto de Flutter	Gota de miel con mora: legacy (5 densidades), adaptativo Android 8+ y 15 tamaños iOS
Splash Screen	Fondo blanco vacío	Fondo mora + logo: Android < 12, Splash API Android 12+, LaunchScreen iOS y splash animado en Flutter

Elemento	Antes · Dulce Antojo (AI 1)	Después · Mora & Miel (AI 2)
Paleta / tema	`ThemeData(colorSchemeSeed: Colors.pink)` y `Colors.pink` repetido	Tema Material 3 central (`AppTheme`) con 7 colores de marca
Pantallas	2 (Home, Detalle)	4 (Splash, Home, Detalle con foto, Mis reservas) + banner global
Arquitectura	Widgets con datos en una lista global	Capas: presentación, estado (Provider), servicios nativos y datos
Persistencia	Ninguna	Caché del catálogo y reservas en SharedPreferences; fotos en la carpeta privada de la app
Servicios nativos	Ninguno	Conectividad reactiva, cámara/galería con permisos, notificaciones locales, MethodChannel propio

1.4 Comparación de assets

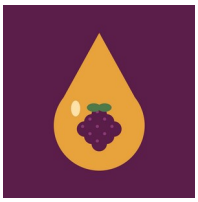
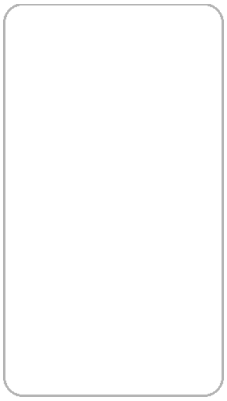
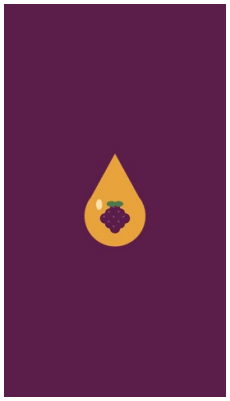
ANTES	DESPUÉS
 Ícono genérico de Flutter	 Ícono Mora & Miel
 Splash: fondo blanco sin marca	 Splash: fondo mora + logo

Figura 1. Ícono y splash antes y después del rebranding (assets reales del repositorio).

1.5 Paleta de colores unificada

Mora	Mora claro	Miel	Crema	Cacao	Hoja	Alerta
#5B1E4B	#8E3B78	#E8A33D	#FFF8EE	#2E1A24	#4F7A4A	#B3261E
AppBar, botones, splash	Enlaces, chips	Acentos, precios	Fondo	Texto	En línea / confirmado	Offline / errores

Figura 2. Paleta definida en lib/core/theme/app_colors.dart y replicada en res/values/colors.xml.

1.6 Interfaz antes vs. después

[INSERTAR CAPTURA] ANTES: pantalla principal de Dulce Antojo (Actividad 1). Puede reutilizar la captura del informe anterior.	**[INSERTAR CAPTURA]** DESPUÉS: pantalla principal de Mora & Miel en el emulador.

Figura 3. Pantalla principal antes y después del rebranding.

Sección II: Arquitectura y Patrones de Desarrollo

La aplicación se reorganizó en **cinco capas** con dependencias en un solo sentido (de arriba hacia abajo). La gestión de estado usa el patrón **Provider + ChangeNotifier**, y todos los servicios nativos viven en `lib/services/` como clases Dart puras, **sin BuildContext ni widgets**, lo que los desacopla por completo de la interfaz.

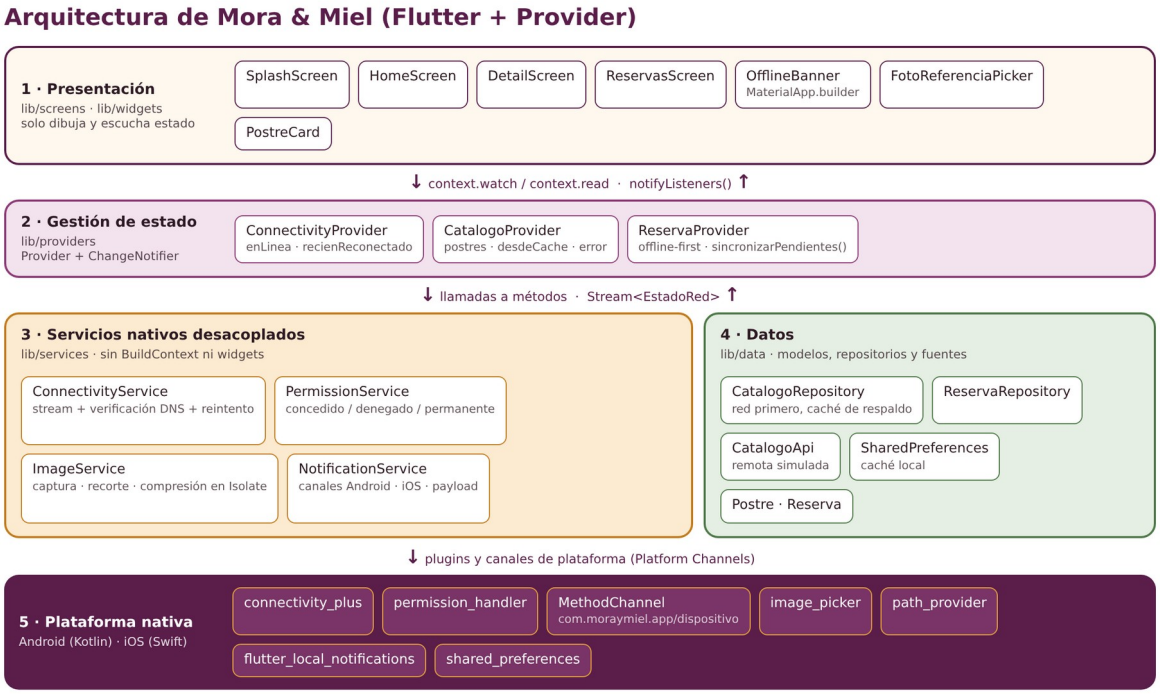


Figura 4. Diagrama de arquitectura: capas de presentación, estado, servicios nativos, datos y plataforma.

2.1 Responsabilidad de cada capa

Capa	Carpeta	Responsabilidad
1. Presentación	`lib/screens`, `lib/widgets`	Dibuja la interfaz y reacciona al estado con `context.watch`. No conoce plugins ni almacenamiento.
2. Estado	`lib/providers`	`ConnectivityProvider`, `CatalogoProvider` y `ReservaProvider` (ChangeNotifier). Orquestan servicios y repositorios y llaman a `notifyListeners()`.
3. Servicios nativos	`lib/services`	Red (`ConnectivityService`), permisos (`PermissionService`), imágenes (`ImageService`) y notificaciones (`NotificationService`). Se comunican con la capa de estado mediante métodos y Streams.
4. Datos	`lib/data`	Modelos (`Postre`, `Reserva`), fuente remota simulada (`CatalogoApi`) y repositorios con caché en SharedPreferences.
5. Plataforma	plugins + Kotlin/Swift	connectivity_plus, permission_handler, image_picker, path_provider, flutter_local_notifications y un MethodChannel propio en `MainActivity.kt`.

2.2 Cómo se logra el desacoplamiento

- **Inyección de dependencias con MultiProvider** (`main.dart`): los servicios se crean una sola vez y se entregan a los providers por constructor. Para cambiar una implementación (por ejemplo, un servicio falso en pruebas) solo se modifica `main.dart`.
- **Red:** `ConnectivityService` traduce el plugin a un `Stream<EstadoRed>` propio. `CatalogoProvider` y `ReservaProvider` se suscriben a ese stream; el `OfflineBanner` solo lee `ConnectivityProvider`. Ningún widget importa `connectivity_plus`.
- **Cámara:** el widget `FotoReferenciaPicker` pide el permiso a `PermissionService` y la imagen a `ImageService`; recibe un resultado tipado (`ResultadoPermiso`, `ImagenProcesada`) y solo decide qué mostrar.
- **Notificaciones:** `ReservaProvider` dispara eventos de dominio (`reservaConfirmada`, `reservaGuardadaOffline`, `reservasSincronizadas`) y `NotificationService` decide canal, ícono e importancia. La navegación al tocar la notificación usa un `navigatorKey` global, sin depender de un BuildContext.

2.3 Flujo de una reserva sin conexión

#	Componente	Acción
1	ConnectivityService	El stream nativo reporta `none` → emite `EstadoRed.sinConexion`.
2	OfflineBanner	Se reconstruye y muestra el banner rojo “Sin conexión”.
3	DetailScreen → ReservaProvider	`crearReserva()` detecta que no hay red y guarda la reserva como **pendiente** en SharedPreferences.
4	NotificationService	Muestra “Reserva guardada sin conexión” en el canal *reservas*.
5	ConnectivityService	Vuelve la red y la verificación DNS es exitosa → emite `EstadoRed.enLinea`.
6	ReservaProvider	`sincronizarPendientes()` envía las pendientes, las marca **confirmadas** y lanza “Reservas sincronizadas”.
7	CatalogoProvider	Como mostraba la caché, recarga el catálogo desde la red automáticamente.

Sección III: Explicación de Código y Manejo de Excepciones

Los extractos siguientes se copiaron del código fuente del repositorio (rama develop/main).

3.1 Detección reactiva de red (ConnectivityService)

lib/services/connectivity_service.dart · _iniciar()

```
Future<void> _iniciar() async {
  try {
    final inicial = await _connectivity.checkConnectivity();
    _emitir(await _evaluar(inicial));
  } catch (_) {
    // Si el plugin falla (p. ej. en un test), asumimos en línea.
    _emitir(EstadoRed.enLinea);
  }

  // asyncMap procesa los eventos en orden: un evento viejo no puede
  // sobrescribir a uno nuevo aunque su verificación DNS tarde más.
  _suscripcion = _connectivity.onConnectivityChanged
      .asyncMap(_evaluar)
      .listen(_emitir);
}
```

- `checkConnectivity()` obtiene el estado inicial al abrir la app; si el plugin lanza una excepción (por ejemplo en pruebas) se asume “en línea” para no bloquear al usuario.
- `onConnectivityChanged` es un **Stream nativo** (NetworkCallback en Android, NWPathMonitor en iOS) que emite cada vez que cambia la red.
- `asyncMap(_evaluar)` procesa los eventos **en orden**: si llegan dos cambios seguidos, el resultado del primero nunca sobrescribe al segundo aunque su verificación tarde más.

lib/services/connectivity_service.dart · _evaluar()

```
Future<EstadoRed> _evaluar(List<ConnectivityResult> resultados) async {
  _reintento?.cancel();
  final sinInterfaz =
      resultados.isEmpty ||
      resultados.every((r) => r == ConnectivityResult.none);
  if (sinInterfaz) return EstadoRed.sinConexion;

  final hayInternet = await _hayInternetReal();
  if (!hayInternet) {
    // La red puede tardar unos segundos en estar lista (p. ej. al quitar
    // el modo avión). Volvemos a comprobar sin esperar otro evento.
    _reintento = Timer(const Duration(seconds: 5), () async {
      if (await _hayInternetReal()) _emitir(EstadoRed.enLinea);
    });
  }
  return hayInternet ? EstadoRed.enLinea : EstadoRed.sinConexion;
}
```

- Tener Wi-Fi no significa tener internet (portal cautivo, router sin salida). Por eso, si hay interfaz, se hace una **consulta DNS** con límite de 3 s.
- Si la consulta falla justo después de quitar el modo avión, se programa un **reintento con Timer** a los 5 s; así el banner no queda “atascado” en offline.

Manejo de excepciones de red

```
Future<bool> _hayInternetReal() async {
  try {
    final r = await InternetAddress.lookup(
      'example.com',
    ).timeout(const Duration(seconds: 3));
    return r.isNotEmpty && r.first.rawAddress.isNotEmpty;
  } on SocketException {
    return false;
  } on TimeoutException {
    return false;
  }
}
```

`SocketException` (sin ruta o DNS caído) y `TimeoutException` (red lenta) se capturan y se convierten en `false`: el error de red **nunca** llega a la interfaz como un fallo, solo como el estado “sin conexión”.

3.2 Estrategia de caché y sincronización

lib/data/repositories/catalogo_repository.dart

```
Future<CatalogoResultado> obtener({required bool hayInternet}) async {
  final prefs = await SharedPreferences.getInstance();

  if (hayInternet) {
    try {
      final postres = await api.obtenerPostres();
      final ahora = DateTime.now();
      await prefs.setString(
        _kCache,
        jsonEncode(postres.map((p) => p.toJson()).toList()),
      );
      await prefs.setString(_kFecha, ahora.toIso8601String());
      return CatalogoResultado(postres, desdeCache: false, actualizado: ahora);
    } catch (_) {
      // Si la red falla a mitad de camino, seguimos con la caché.
    }
  }

  final raw = prefs.getString(_kCache);
  if (raw == null) {
    throw const CatalogoSinDatosException();
  }
```

Estrategia **“red primero, caché de respaldo”**: con internet se descarga y se guarda una copia en SharedPreferences; si no hay red o la API falla (`catch`), se devuelve la copia guardada. Si es el primer uso y no existe copia, se lanza la excepción propia `CatalogoSinDatosException`, que `CatalogoProvider` muestra con un botón *Reintentar*.

lib/providers/reserva_provider.dart · sincronizarPendientes()

```
Future<int> sincronizarPendientes() async {
  // Bandera para evitar dos sincronizaciones simultáneas (concurrencia).
  if (_sincronizando || pendientes == 0) return 0;
  _sincronizando = true;
  var enviadas = 0;
  try {
    for (var i = 0; i < _reservas.length; i++) {
      if (_reservas[i].estado != EstadoReserva.pendiente) continue;
      if (!_red.enLinea) break; // se volvió a caer la red
      await _enviarAlServidor();
      _reservas[i] = _reservas[i].copyWith(estado: EstadoReserva.confirmada);
      enviadas++;
    }
    if (enviadas > 0) {
      await _repo.guardarTodas(_reservas);
      notifyListeners();
      await _notificaciones.reservasSincronizadas(enviadas);
    }
  } finally {
    _sincronizando = false;
  }
  return enviadas;
}
```

La bandera `_sincronizando` evita que dos eventos “en línea” seguidos envíen la misma reserva dos veces; `try/finally` garantiza que la bandera se libere aunque ocurra un error. Si la red se vuelve a caer a mitad del proceso (`break`), las restantes quedan pendientes para el siguiente intento.

3.3 Manejo dinámico de permisos (PermissionService)

android/.../MainActivity.kt · MethodChannel propio (Kotlin)

```
    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CANAL)
            .setMethodCallHandler { call, result ->
                when (call.method) {
                    "sdkInt" -> result.success(Build.VERSION.SDK_INT)
                    else -> result.notImplemented()
                }
            }
```

Android 13 cambió el permiso de galería (`READ_MEDIA_IMAGES`) respecto a Android 12 o anterior (`READ_EXTERNAL_STORAGE`). Para saber cuál pedir, se creó un **canal de plataforma propio** que devuelve `Build.VERSION.SDK_INT` desde Kotlin.

lib/services/permission_service.dart · solicitar()

```
Future<ResultadoPermiso> solicitar(TipoPermiso tipo) async {
  final permiso = await _permisoDe(tipo);

  // Si ya está concedido no molestamos al usuario.
  final actual = await permiso.status;
  if (actual.isGranted || actual.isLimited) return ResultadoPermiso.concedido;
  if (actual.isRestricted) return ResultadoPermiso.restringido;

  // En Android el "denegado permanente" solo se detecta de forma fiable
  // con el resultado de request(): si ya fue bloqueado, el sistema no
  // muestra diálogo y devuelve permanentlyDenied de inmediato.
  final resultado = await permiso.request();
  return _mapear(resultado);
}
```

- Si el permiso ya está concedido (o limitado en iOS) no se vuelve a mostrar el diálogo.
- En Android el estado “denegado permanentemente” solo es fiable con el resultado de `request()`: si el usuario marcó “No volver a preguntar”, el sistema responde de inmediato sin diálogo.
- El resultado se traduce a un enum propio `ResultadoPermiso` (concedido, denegado, denegadoPermanente, restringido) para que la UI no dependa del plugin.

lib/widgets/foto_referencia_picker.dart · control de permisos denegados

```
  switch (resultado) {
    case ResultadoPermiso.denegado:
      _mensaje('Permiso denegado: no podemos acceder a $recurso.');
    case ResultadoPermiso.restringido:
      _mensaje('El acceso a $recurso está restringido en este dispositivo.');
    case ResultadoPermiso.denegadoPermanente:
      final abrir = await showDialog<bool>(
        context: context,
        builder: (ctx) => AlertDialog(
          icon: const Icon(Icons.lock_outline, color: AppColors.mora),
          title: const Text('Permiso desactivado'),
          content: Text(
            'Negaste el acceso a $recurso de forma permanente. '
            'Para adjuntar una foto de referencia actívalo en Ajustes.',
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(ctx, false),
              child: const Text('Ahora no'),
            ),
            ElevatedButton(
              onPressed: () => Navigator.pop(ctx, true),
              child: const Text('Abrir Ajustes'),
            ),
          ],
        ),
      );
      if (abrir == true) await permisos.abrirAjustes();
```

Cada caso tiene una respuesta distinta: **denegado** → aviso en SnackBar (se puede reintentar); **restringido** → aviso de bloqueo del sistema; **denegado permanente** → diálogo que explica el motivo y abre los Ajustes de la app con `openAppSettings()`.

3.4 Captura, recorte y compresión (ImageService)

lib/services/image_service.dart · función ejecutada en un Isolate

```
({Uint8List jpg, int ancho, int alto}) _recortarYComprimir(
  (Uint8List, bool) args,
) {
  final (bytes, cuadrado) = args;
  final original = img.decodeImage(bytes);
  if (original == null) {
    throw const FormatException('Formato de imagen no soportado');
  }

  // Corrige la rotación EXIF (fotos de cámara giradas).
  var imagen = img.bakeOrientation(original);

  if (cuadrado) {
    final lado = math.min(1080, math.min(imagen.width, imagen.height));
    imagen = img.copyResizeCropSquare(
      imagen,
      size: lado,
      interpolation: img.Interpolation.average,
    );
  } else if (imagen.width > 1280) {
```

- Primer nivel de compresión en el lado nativo: `pickImage(maxWidth: 2048, imageQuality: 90)`.
- Segundo nivel en Dart: corrección de rotación EXIF (`bakeOrientation`), recorte cuadrado de hasta 1080 px y JPG calidad 75. Se ejecuta con `compute()` en **otro Isolate** para no congelar la animación de carga.
- Excepciones: imagen corrupta → `FormatException` con mensaje claro; cualquier otro error (sin espacio, cámara ocupada) se captura en el widget y se muestra en un SnackBar. Si Android destruye la Activity mientras la cámara está abierta, `retrieveLostData()` recupera la foto al volver.

3.5 Servicio de notificaciones locales (NotificationService)

lib/services/notification_service.dart · canales nativos

```
  static const canalReservas = AndroidNotificationChannel(
    'reservas',
    'Reservas',
    description: 'Confirmación y estado de tus reservas en Mora & Miel',
    importance: Importance.high,
  );

  /// Canal para avisos de sincronización / conexión.
  static const canalSincronizacion = AndroidNotificationChannel(
    'sincronizacion',
    'Sincronización',
    description: 'Avisos cuando se envían reservas guardadas sin conexión',
    importance: Importance.defaultImportance,
  );

  /// Payload que indica abrir la pantalla "Mis reservas".
  static const payloadReservas = 'reservas';
```

lib/services/notification_service.dart · iniciar()

```
Future<void> iniciar() async {
  if (_iniciado) return;

  const ajustes = InitializationSettings(
    // Ícono monocromo propio (res/drawable-*/ic_stat_mora.png).
    android: AndroidInitializationSettings('ic_stat_mora'),
    // En iOS el permiso se pide después, en un momento oportuno.
    iOS: DarwinInitializationSettings(
      requestAlertPermission: false,
      requestBadgePermission: false,
      requestSoundPermission: false,
    ),
  );

  await _plugin.initialize(
    settings: ajustes,
    onDidReceiveNotificationResponse: _alTocarNotificacion,
  );

  final android = _plugin
      .resolvePlatformSpecificImplementation<
        AndroidFlutterLocalNotificationsPlugin
      >();
  await android?.createNotificationChannel(canalReservas);
  await android?.createNotificationChannel(canalSincronizacion);

  _iniciado = true;
}
```

- Desde Android 8 toda notificación debe pertenecer a un **canal**; se crean dos con importancias distintas: *reservas* (alta, aparece como heads-up) y *sincronizacion* (normal). El usuario puede silenciarlos por separado desde Ajustes.
- Se usa un ícono monocromo propio `ic_stat_mora` (Android exige silueta blanca) protegido del shrinker con `res/raw/keep.xml`.
- En iOS el permiso no se pide al iniciar sino en `solicitarPermiso()`, y `AppDelegate.swift` asigna el delegado de `UNUserNotificationCenter` para mostrar alertas con la app en primer plano.

lib/services/notification_service.dart · _mostrar()

```
Future<void> _mostrar({
  required AndroidNotificationChannel canal,
  required String titulo,
  required String cuerpo,
}) async {
  try {
    await iniciar();
    final detalles = NotificationDetails(
      android: AndroidNotificationDetails(
        canal.id,
        canal.name,
        channelDescription: canal.description,
        importance: canal.importance,
        priority: canal.importance == Importance.high
            ? Priority.high
            : Priority.defaultPriority,
        color: AppColors.mora,
        styleInformation: BigTextStyleInformation(cuerpo),
      ),
      iOS: const DarwinNotificationDetails(
        presentAlert: true,
        presentBanner: true,
        presentSound: true,
      ),
    );
    await _plugin.show(
      id: _siguienteId++,
      title: titulo,
      body: cuerpo,
      notificationDetails: detalles,
      payload: payloadReservas,
    );
  } catch (e) {
    // Una notificación fallida nunca debe romper el flujo de la reserva.
    debugPrint('No se pudo mostrar la notificación: $e');
  }
}
```

Toda la publicación está dentro de `try/catch`: si el usuario negó el permiso o el sistema rechaza la notificación, **la reserva igual se guarda** y solo se registra el error con `debugPrint`. El `payload` “reservas” permite abrir la pantalla *Mis reservas* al tocar la notificación, incluso con la app cerrada (`getNotificationAppLaunchDetails`).

3.6 Resumen de excepciones controladas

Situación	Dónde	Tratamiento
Sin interfaz de red / DNS falla / timeout	ConnectivityService	Se emite `sinConexion`; reintento a los 5 s.
API falla o no hay red	CatalogoRepository	Se usa la caché local.
Primer uso sin red y sin caché	CatalogoRepository → Provider	`CatalogoSinDatosException` + botón Reintentar.
Doble sincronización simultánea	ReservaProvider	Bandera `_sincronizando` + `try/finally`.
Permiso denegado / permanente / restringido	PermissionService + widget	SnackBar o diálogo “Abrir Ajustes”.

Situación	Dónde	Tratamiento
Usuario cancela cámara/galería	FotoReferenciaPicker	`pickImage` devuelve `null` → no se hace nada.
Imagen corrupta o no soportada	ImageService (Isolate)	`FormatException` con mensaje claro.
Activity destruida con la cámara abierta	ImageService	`retrieveLostData()` recupera la foto.
Notificación rechazada / sin permiso	NotificationService	`try/catch`; la reserva no se pierde.
Widget desmontado tras un `await`	Pantallas / widgets	Comprobación de `mounted` antes de usar el contexto.

Sección IV: Evidencias de Pruebas (QA) y Enlace al Repositorio Público

4.1 Repositorio público

URL: https://github.com/<usuario>/mora-y-miel-flutter

Evidencia	Detalle
Commits	10 commits de trabajo + merges, con mensajes estilo Conventional Commits (feat, refactor, style, docs, chore).
Ramas	`main` (estable), `develop` (integración), `feature/rebranding`, `feature/arquitectura-provider`, `feature/conectividad`, `feature/camara-permisos`, `feature/notificaciones`.
Etiquetas	`v2.0.0-branding` (rebranding estable en main) y `v2.0.0` (release final).
README.md	Descripción, cuadro de rebranding, servicios nativos, diagrama de arquitectura, instalación/ejecución, guía de pruebas y flujo de ramas.
Calidad	`flutter analyze`: sin errores ni advertencias (reglas flutter_lints).

4.2 Historial de ramas (git log --graph)

git log --graph --oneline --all --decorate

```
*   a5c85be (tag: v2.0.0, main) release: v2.0.0 (Mora & Miel con servicios nativos)
|\
| * 907d41a (HEAD -> develop) docs: README profesional, diagrama de arquitectura e imágenes de rebranding
| *   32708e2 merge: feature/notificaciones en develop
| |\
| | * 0ac6469 (feature/notificaciones) feat(notificaciones): notificaciones locales con canales Android e iOS
| |/
| *   46c9881 merge: feature/camara-permisos en develop
| |\
| | * 7b8a619 (feature/camara-permisos) feat(camara): cámara/galería con permisos en tiempo de ejecución, recorte y compresión
| |/
| *   465e8a3 merge: feature/conectividad en develop
| |\
| | * 73576bb (feature/conectividad) feat(red): detección reactiva de conectividad, banner offline y sincronización
| |/
| *   a980ca1 merge: feature/arquitectura-provider en develop
| |\
| | * 320cbea (feature/arquitectura-provider) refactor(arquitectura): capas data/providers con Provider y caché local
| |/
* | c1232b7 (tag: v2.0.0-branding) release: v2.0.0-branding (rebranding estable Mora & Miel)
|\|
| * 52cc93a merge: feature/rebranding en develop
|/|
| * 33e43dd (feature/rebranding) feat(branding): nuevo ícono de app y splash screen de la marca
| * f10513f style(branding): nueva paleta "Mora & Miel" y tema centralizado
| * d2da1c3 refactor(branding): renombrar app a "Mora & Miel" y nuevo package id
|/
* 4a396f8 chore: proyecto base de la Actividad Integradora 1 (Dulce Antojo) y .gitignore
```

[INSERTAR CAPTURA]
Captura de la página del repositorio en GitHub mostrando las ramas (Branches) o la gráfica de commits (Insights → Network).

Figura 5. Ramas y commits en el repositorio público.

4.3 Casos de prueba ejecutados

ID	Caso de prueba	Resultado esperado	Estado
QA-01	Abrir la app (splash nativo + animado)	Fondo mora con logo y transición al catálogo	Aprobado
QA-02	Activar modo avión en el catálogo	Banner rojo y catálogo desde caché	Aprobado
QA-03	Desactivar modo avión	Banner verde y recarga automática	Aprobado
QA-04	Reservar sin conexión	Reserva “Pendiente” + notificación offline	Aprobado
QA-05	Reconectar con pendientes	Pasa a “Confirmada” + notificación de sincronización	Aprobado
QA-06	Pulsar “Cámara” por primera vez	Diálogo de permiso del sistema	Aprobado
QA-07	Negar permiso con “No volver a preguntar”	Diálogo “Abrir Ajustes”	Aprobado
QA-08	Tomar/elegir foto	Previsualización 1:1 con KB antes/después	Aprobado
QA-09	Reservar con internet	Notificación “¡Reserva confirmada!”	Aprobado
QA-10	Tocar la notificación	Se abre “Mis reservas”	Aprobado

Revise la columna “Estado” y actualícela según sus pruebas reales en el emulador o dispositivo.

4.4 Capturas de funcionamiento

Dispositivo / emulador: [p. ej. Pixel 8 – Android 15 (API 35), emulador de Android Studio]

[INSERTAR CAPTURA]
QA-01: splash screen de Mora & Miel.

[INSERTAR CAPTURA]
QA-02: catálogo en MODO OFFLINE con el banner rojo “Sin conexión” y el texto “Mostrando catálogo guardado”.

Figura 6. Splash screen y modo offline.

[INSERTAR CAPTURA]
QA-03: banner verde “Conexión restablecida”.

[INSERTAR CAPTURA]
QA-04/05: pantalla “Mis reservas” con una reserva Pendiente (y luego Confirmada).

Figura 7. Reconexión y sincronización de reservas.

[INSERTAR CAPTURA]
QA-06: diálogo del sistema solicitando PERMISO DE CÁMARA.

[INSERTAR CAPTURA]
QA-07: diálogo “Permiso desactivado → Abrir Ajustes” tras negar el permiso.

Figura 8. Solicitud y control de permisos en tiempo de ejecución.

[INSERTAR CAPTURA]
QA-08: previsualización de la foto recortada y comprimida (KB antes → después).

[INSERTAR CAPTURA]
QA-09: NOTIFICACIÓN recibida “¡Reserva confirmada!” en la barra de notificaciones.

Figura 9. Procesamiento de imagen y recepción de notificaciones.

[INSERTAR CAPTURA]
Notificación “Reserva guardada sin conexión” o “Reservas sincronizadas”.

[INSERTAR CAPTURA]
Ajustes del sistema → Notificaciones de Mora & Miel mostrando los canales “Reservas” y “Sincronización”.

Figura 10. Notificaciones de los eventos offline y canales nativos de Android.

Sección V: Conclusiones y Lecciones Aprendidas

5.1 Ciclo de vida del dispositivo

- **Los recursos nativos deben liberarse.** Cada suscripción a un Stream (`ConnectivityService`, providers) se cancela en `dispose()`; de lo contrario el listener nativo sigue activo y provoca fugas de memoria y llamadas sobre objetos destruidos.
- **Android puede matar la app en cualquier momento.** Al abrir la cámara, la Activity pasa a segundo plano y el sistema puede destruirla por falta de memoria; por eso se implementó `retrieveLostData()`. También aprendí que las reservas y la caché deben persistir en disco y no solo en memoria.
- **El contexto puede dejar de existir tras un await.** Comprobar `mounted` antes de usar `context`, y diferir la carga inicial con `addPostFrameCallback`, evita errores como “setState() called during build” o usar un widget ya desmontado.
- **Los permisos dependen de la versión del sistema.** Android 13 cambió los permisos de fotos y agregó `POST_NOTIFICATIONS`; iOS exige textos de uso en `Info.plist`. Un mismo botón necesita lógica distinta según la plataforma, y un canal de plataforma propio resultó una solución simple para consultar la versión.

5.2 Concurrencia y asincronía

- **Isolates para trabajo pesado.** Decodificar y recomprimir una foto de varios MB en el hilo principal congela la interfaz; con `compute()` el procesamiento ocurre en paralelo y la UI sigue respondiendo.
- **El orden de los eventos importa.** `asyncMap` garantiza que los cambios de red se procesen en orden; `Future.wait` permite cargar catálogo, reservas y notificaciones en paralelo durante el splash, reduciendo el tiempo de arranque.
- **Condiciones de carrera.** Dos eventos “en línea” muy seguidos podían sincronizar dos veces la misma reserva; una bandera con `try/finally` lo resolvió. La programación reactiva obliga a pensar qué pasa si algo ocurre dos veces o en otro orden.

5.3 Control de versiones

- **Ramas por funcionalidad.** Trabajar cada servicio en su `feature/*` permitió aislar cambios grandes (por ejemplo, el renombrado del package) y mantener `main` siempre estable con el rebranding.
- **Merges con --no-ff y etiquetas.** Conservan en el historial cuándo se integró cada funcionalidad y permiten volver a una versión exacta (`v2.0.0-branding`, `v2.0.0`).
- **Commits pequeños y descriptivos** (Conventional Commits) hacen el proyecto auditable: el docente o un compañero puede revisar qué cambió en cada paso y por qué, y un `.gitignore` correcto evita subir archivos generados como `local.properties`.

5.4 Conclusión general

La reingeniería demostró que una aplicación móvil profesional no es solo su interfaz: separar la lógica nativa en servicios, centralizar la identidad visual y documentar cada cambio en el repositorio hace que la app sea más robusta (funciona sin internet, controla permisos y errores), más fácil de mantener y verificable por terceros. El cambio de “Dulce Antojo” a “Mora & Miel” pasó de un prototipo de dos pantallas a un producto con una propuesta de valor clara.

Declaración de uso de Inteligencia Artificial

En el desarrollo de esta actividad se utilizó el asistente de IA **Claude (Anthropic)** como apoyo para: proponer la estructura por capas, generar el código base de los servicios nativos, el ícono y el splash, redactar el README y el borrador de este informe. Todo el código fue revisado, ejecutado y probado por la estudiante en su entorno, y los commits generados con apoyo de la IA incluyen la etiqueta `Co-Authored-By` para mantener la transparencia.

Ajuste este párrafo para que describa con exactitud cómo usó la IA y qué partes modificó o verificó usted.